A New Flexible Algorithm for the Longest Common Subsequence Problem*

Claus Rick

University of Bonn, Computer Science Department IV,
Römerstr. 164, 53117 Bonn, Germany, Email: rick@cs.uni-bonn.de

Abstract. A new algorithm that is efficient for both short and long longest common subsequences is presented. It also improves on previous algorithms for longest common subsequences of intermediate length. Thus, it is more flexible and can be used for a wider range of applications than others. The algorithm is based on the well-known paradigm of computing dominant matches and was obtained through a kind of dualization. Some experimental results are given, too.

1 Introduction

Let $A = a_1a_2 \dots a_m$ and $B = b_1b_2 \dots b_n, m \leq n$, be two sequences over an alphabet Σ of size s. A sequence $S = s_1s_2 \dots s_l, l \leq m$, is called a *common subsequence* of A and B iff S can be obtained from both A and B by deleting a number of (not necessarily consecutive) symbols. Finding a common subsequence of greatest possible length is called the *longest common subsequence* (LCS) *problem*. Let $|A|$ denote the length of a string A and let $A_i, 0 \leq i \leq |A|$, denote the length i prefix of A. Define $L_{i,j} = \max\{|S| \ : \ S \text{ is a common subsequence of } A_i \text{ and } B_j\}$, $0 \leq i \leq m, 0 \leq j \leq n$, to be the length of a LCS between prefixes A_i and B_j.

Then the LCS problem can be solved by a dynamic programming approach [14, 16]. It works on the $(m+1) \times (n+1)$ L-matrix spanned up by the two strings by computing all the values $L_{i,j}$ using the recursion

$$L_{i,j} = \begin{cases} 0 & \text{if} \quad i = 0 \text{ or } j = 0 \\ L_{i-1,j-1} + 1 & \text{if} \quad a_i = b_j \\ \max\{L_{i-1,j}, L_{i,j-1}\} & \text{if} \quad a_i \neq b_j \end{cases}$$

in time and space $\Theta(mn)$. The asymptotically fastest general solution needs time $O(n^2/\log n)$ [9] and uses the "Four Russians" trick. A lot of algorithms have been developed that, although not improving the general $O(mn)$ time bound of the dynamic programming approach, exhibit a much better performance by specializing on certain classes of pairs of sequences. These algorithms are sensitive to other problem parameters, e.g. the length p of a LCS (the output parameter), the number of (dominant) matches or the alphabet size s (see [12] for a survey).

* This work is dedicated to my father and to the memory of my mother.

Generally speaking, there are algorithms that suit for a class of pairs of sequences where a LCS can be expected to be very long. Their time complexity is $O(n(n-p))$ or $O(n(m-p))$. On the other hand, there are algorithms that are fast for short longest common subsequences. Their running time is roughly $O(pm)$. As pointed out in [11] we have to select one of the algorithms a priori depending on the kind of sequences we wish to compare. However, this might prove difficult because of insufficient knowledge of the nature of the sequences to be compared. Or the class of sequences is so heterogeneous that both short and long longest common subsequences are likely to occur, e.g. when searching a data base for entries similar to a query. Moreover, when dealing with sequences of approximately equal length consisting of symbols drawn from a small alphabet in a more or less uniform manner the length of a LCS can be expected to lie in the range between $\frac{1}{3}m$ and $\frac{2}{3}m$, depending on the alphabet size [4, 12, 14]. This situation is typical of the analysis of macromolecular sequences in biology. None of the algorithms mentioned above is well-suited for the cases described. Although some algorithms exist whose time analysis indicates somewhat more flexibility [1, 5, 6, 7], the constant factors employed by these methods seem rather high. No specific attention was given to the above problems so far.

In this paper, a new algorithm for the LCS problem is presented which exhibits a good performance for both short and long longest common subsequences and can deal with longest common subsequences of intermediate length in a more efficient manner than previous algorithms. The worst case running time of the algorithm is $O(ns + \min\{pm, p(n-p)\})$. Experimental results proved its practicability.

2 The Paradigm of Computing Dominant Matches

There are two main paradigms used by all the algorithms for the LCS problem. One is to compute a cheapest-cost path in an editgraph [10, 15, 17]. The other amounts to what Eppstein et al. [5] call sparse dynamic programming. It will be reviewed since it is the basis for the new algorithm, too. An ordered pair (i, j) of positions in A and B is called a *match* iff $a_i = b_j$, $1 \leq i \leq |A|, 1 \leq j \leq |B|$. Obviously the LCS problem can now be restated as the problem of finding a longest sequence of matches that is strictly increasing in both components. Thus, it suffices to compute the entries $L_{i,j}$ in the L-matrix where (i, j) is a match. For each match the value $L_{i,j}$ is called its *rank*. A match of rank k is said to be a k-match and k is the highest position in a common subsequence to which this match may correspond. We can collect matches of the same rank k in *classes* $C_k := \{(i, j) \mid a_i = b_j \wedge L_{i,j} = k\}$, $k \geq 0$, where $C_0 := \{(0, 0)\}$. The goal is, starting with C_0, to determine the ranks of more and more matches with the aid of matches whose rank is already known. A match (i, j) is called a *successor (predecessor)* of match (i', j') if $i' < i \ \wedge \ j' < j$ $(i' > i \ \wedge \ j' > j)$. The following observations are easy to see. For each match in class C_k there has to be a (*generating*) predecessor in the preceding class C_{k-1}. A match must have greater rank than each of his predecessors and therefore matches belonging

to the same class cannot be successors or predecessors of each other. Figurally speaking, matches of the same class shift from right to left in the L-matrix for increasing rows and contours given by connecting matches of the same class do never cross or touch. A lot of matches are superfluous in the sense that they are not essential for the generation of matches in the next higher class. In fact it is well-known that we can concentrate on the so-called *dominant* matches, i.e. the minimal matches in each class according to the standard "$\leq$" vector ordering; there is always a LCS consisting solely of dominant matches. In the L-matrix these are the matches in each class where no other match of the same class lies to the left in the same row or above in the same column. Now denote the smallest column of the k-matches up to the i-th row by $T_{i,k}$, i. e.

$$T_{i,k} := \min\{\{t \mid (s,t) \in C_k,\ 0 \leq s \leq i\} \cup \{\infty\}\}, \quad 0 \leq i \leq m,\ k \geq 0\ .$$

The main observation is that a match (i,j) in row i is in C_k if and only if $T_{i-1,k-1} < j \leq T_{i-1,k}$. Notice that a match with $j \leq T_{i-1,k-1}$ would have no predecessor in C_{k-1} and that $j > T_{i-1,k}$ would imply a rank of at least $k+1$ for (i,j). We can therefore state a recursion for $T_{i,k}$:

$$T_{i,k} = \begin{cases} \min\{j \mid a_i = b_j \wedge T_{i-1,k-1} < j \leq T_{i-1,k}\} \\ T_{i-1,k} \text{ if there is no such } j \end{cases} .$$

Note that $T_{i,k} < T_{i-1,k}$ implies that $(i, T_{i,k})$ is a dominant match in C_k. We even have that (i,j) is minimal in C_k if and only if $T_{i-1,k-1} < j < T_{i-1,k}$ and $j = T_{i,k}$. Initializing $T_{0,0}$ by 0 and $T_{0,k}$ by ∞, $k \geq 1$, we can compute all dominant matches row by row using the recursion above. Instead of defining $T_{i,k}$ as the smallest column of the k-matches up to row i we could also define $T_{j,k}$ as the smallest row of the k-matches up to column j and then, after making analogous observations, compute the dominant matches column by column.

Let us now look at an algorithm presented by Apostolico and Guerra [1] implementing this paradigm. They observed that it is very useful to precompute information about the first occurrence of a symbol in sequence $B = b_1 b_2 \ldots b_n$ to the right of some given position. Let $\Sigma = \{\sigma_1, \sigma_2, \ldots, \sigma_s\}$ be the relevant alphabet and define

$$\begin{aligned} &CLOSEST : [\sigma_1, \ldots, \sigma_s] \times [0, \ldots, n+1] \longrightarrow [1, \ldots, n+1] \\ &CLOSEST \quad [\sigma_i, j] := \min\{\{j' > j \mid b_{j'} = \sigma_i\} \cup \{n+1\}\}\ . \end{aligned}$$

If s is fixed and small, this function can be made available in form of a $s \times (n+1)$ matrix computable in time and space $O(sn)$. A compact representation can be computed in time and space $O(n)$ at the cost of $O(\log s)$ time for each lookup. Assume that we would like to compute a dominant k-match (i,j) in row i. Looking up $j := CLOSEST[a_i, T_{i-1,k-1}]$ we have three cases:

1. $j < T_{i-1,k} \Longrightarrow (i,j)$ is a dominant k-match;
2. $j = T_{i-1,k} \Longrightarrow (i,j) \in C_k$, but is no dominant match;
3. $j > T_{i-1,k} \Longrightarrow (i,j)$ is a (dominant) match in the first class $C_{k'}, k' > k$, with $(T_{i-1,k'} > j)\ T_{i-1,k'} = j$.

Employing a row-by-row order of computation the following algorithm uses one vector $Thresh$ of size $O(m)$ to store the current leftmost column of dominant matches found so far for each class respectively. The correctness follows immediately from the preceding discussion and the invariant $Thresh[k] = T_{i-1,k},\ k \geq 0$, at the beginning of the i-th iteration of the for-loop. **Algorithm 1** [1] takes as input the sequences $A = a_1 a_2 \ldots a_m$ and $B = b_1 b_2 \ldots b_n, m \leq n$, and outputs a LCS $S = s_1 s_2 \ldots s_p$.

```
(0)   Preprocessing: compute CLOSEST for sequence B
(1)   Thresh[0] := 0;
      for k := 1 to m do Thresh[k] := n + 1 od;
(2)   for i := 1 to m do;                          (* proceed row by row *)
          j := CLOSEST[a_i, 0];                    (* first match in row i *)
          k := 1;
(3)       while j ≠ n + 1 do                       (* still a match left in row i ? *)
              if j < Thresh[k] then
                  temp := Thresh[k];
                  Thresh[k] := j;
(4)               record new dominant match (i, j);
                  j := CLOSEST[a_i, temp];         (* next relevant match *)
              fi;
              if j = Thresh[k] then j := CLOSEST[a_i, j] fi;
              k := k + 1;
          od;
      od;
(5)   recover LCS S
```

This algorithm's time complexity is $O(ns + pm)$. Time $O(ns)$ is needed for the preprocessing and in each iteration of the for-loop we check no more than $O(p)$ elements in $Thresh$ which equals the number of iterations of the while-loop. For each dominant match we dynamically create a new node and let it point to a generating predecessor (step 4) in order to be able to reconstruct a LCS in step 5 in $O(m)$ time (for details see [8]). Therefore the space complexity is $O(ns + d)$ where d denotes the number of dominant matches.

3 A New Flexible Algorithm

As we have argued above, a new dominant match is found if and only if a certain $Thresh$-value can be reduced in the current row. On the other hand the time complexity of all algorithms using the paradigm of computing dominant matches is given by an upper bound on the number of checks of the $Thresh$-values that are necessary to detect these reductions. Therefore it seems to be a good strategy to try to reduce the number of these checks as far as possible by avoiding unnecessary checks. A closer look at Algorithm 1 reveals the main drawback: once a class C_k has been identified not to be empty, the value $Thresh[k]$ has to be checked in each subsequent row. This means that a class is never recognized

as being determined completely. For short longest common subsequences this does not matter too much since only a few $Thresh$-values have to be checked in each row. But as the longest common subsequences are getting longer the algorithm becomes inefficient. How could we make this general approach work for long longest common subsequences, too?

The first step is to observe that a match $s_k = (i, j) \in C_k$ – which is the k-th match in a LCS of length p between sequences A and B of length m and n respectively – can be sandwiched as follows:

$$k \leq i \leq k + m - p \text{ and } k \leq j \leq k + n - p \ . \tag{1}$$

Otherwise (i, j) could not be a k-match or we could not generate a LCS of length p by appending matches after (i, j). As a direct implication of this observation we would not need to check $Thresh[k]$ in rows $i > k + m - p$ because dominant k-matches that might occur in these rows could never be part of a LCS. But unfortunately we can't use this as a termination criterion for the computation of class C_k since we do not know p a priori; this is the value we would like to determine. Thus, our goal will be to formulate another criterion that does not depend on the knowledge of p although it allows us the identification of classes that need not be taken into consideration any further. Note that the above observation also makes a statement of existence: if there is a LCS of length p then in *each* class C_k there has to be at least one dominant k-match that satisfies (1) and which may be extended to a LCS. We call such a match a *k-witness*.

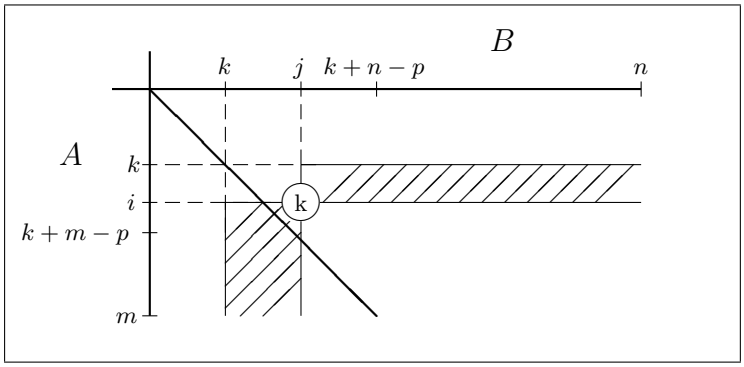


Fig. 1. Assuming a long LCS there are only two small stripes in the L-matrix where we have to search for dominant k-matches; (i, j) shall be a k-witness.

Suppose now the two input sequences are roughly of the same length ($m \approx n$) and there is a long LCS ($p \to m$). Then the components of a k-witness (i, j) must be close to k because of (1). In this case all dominant matches $(i', j') \in C_k$ must lie to the top right or bottom left of this match in the L-matrix, i.e. we either have $i' < i$ or $j' < j$. Of course it must also hold that $i' \geq k$ and $j' \geq k$. Therefore all

dominant matches in C_k must lie in the two small stripes indicated in Figure 1. As a direct consequence each class C_k cannot have more than $(n-p)+(m-p)+1$ dominant matches and so we can give an upper bound on the total number d of dominant matches: $d \leq p(m+n-2p+1)$.

Further we can exploit this observation to devise a new efficient algorithm. Obviously all dominant k-matches have been determined if both stripes of Figure 1 have been searched. Using previous algorithms (which employ a row-by-row order of computation) only matches located in the horizontal stripe will be found quickly. The vertical stripe will be searched completely only after scanning the very last row; dominant matches are computed in order of increasing first components.

Therefore it makes sense to change the order of computation in such a way that dominant matches located in the vertical stripe are searched quickly, too. This can be achieved by computing the dominant matches in order of increasing smaller components, i.e. we first compute dominant matches in row 1 and column 1, followed by those in row 2 and column 2, etc. Of course in row i we only have to consider matches (i,j) with $j \geq i$ (to the right of and on the diagonal) since $j < i$ would imply that the match was found already when considering column j. Analogously, in column j we only have to consider matches (i,j) with $i > j$ (below the diagonal). Let (i_1,j_1), (i_2,j_2), ..., $(i_l,j_l) \in C_k$ be the dominant k-matches computed in this new order. Then we have $k \leq \min\{i_1,j_1\} \leq \min\{i_2,j_2\} \leq \ldots \leq \min\{i_l,j_l\}$. We can now give a sufficient criterion to detect that all dominant k-matches have been computed.

Theorem. *Let $D_{l,k}$ be the set of all dominant k-matches in rows $\leq l$ and columns $\leq l$, i.e. $(i,j) \in D_{l,k} \Rightarrow \min\{i,j\} \leq l$. Then take the minimum among the greater components of these matches as*

$$MinMax_{l,k} := \min\{\max\{i,j\} \mid (i,j) \in D_{l,k}\}$$

and define $l_{min}(k)$ to be the smallest l such that at least one match $(i,j) \in D_{l,k}$ has both components $\leq l$, i.e.

$$l_{min}(k) := \min\{l \mid l = MinMax_{l,k}\} \ .$$

Then we have:

1. *There is no dominant k-match (i',j') with $\min\{i',j'\} > l_{min}(k)$, i.e. the dominant k-matches have been determined completely after searching row and column $l_{min}(k)$;*
2. *$l_{min}(k) \leq k+n-p$, where p is the length of a LCS.*

Proof.

1. According to the definition we have at least one dominant k-match $(i,j) \in D_{l_{min}(k),k}$ with $i \leq l_{min}(k)$ and $j \leq l_{min}(k)$. This match is a predecessor for all matches (i',j') with $\min\{i',j'\} > l_{min}(k)$ which can be no k-matches for this reason. But according to the order of computation only such matches might be found subsequently.

2. Assume that $l_{min}(k) > k + n - p$ would hold. Then no dominant k-match (i, j) such that $i \leq k + m - p \leq k + n - p$ and $j \leq k + n - p$ would exist because at least one component of each dominant k-match would be $> k+n-p$. But this means that there is no k-witness and therefore no LCS of length p. A contradiction. □

Note that this criterion does not depend on the knowledge of p but only on the coordinates of the dominant matches found upto a certain point in the computation. On the other hand we can give an upper bound on the number of rows and columns to be searched for dominant k-matches, namely $(k + n - p) - k + 1$. Assuming that we can detect a dominant k-match in a certain row or column in time $O(1)$ this gives a time bound of $O(n - p + 1)$ to compute all dominant matches of one class and $O(p(n - p + 1))$ to determine all dominant matches in p classes.

The idea to implement the new order of computation consists in a kind of dualization of the method proposed by Apostolico and Guerra which allows us to proceed row by row as well as column by column. We now precompute two tables $CLOSEST\text{-}A$ and $CLOSEST\text{-}B$, at a cost of time and space $O(ns)$, containing information about the next occurrences of symbols in A and B respectively as defined in Section 2. We divide the set of dominant matches into two disjoint subsets of those to the right of and on the diagonal of the L-matrix and those below the diagonal. Intuitively one can imagine to slide down the diagonal, starting at point $(1, 1)$ and ending at point (m, m). Having reached point (l, l), $1 \leq l \leq m$, we can compute dominant matches (l, j), $j \geq l$, in row l with the aid of $CLOSEST\text{-}B$ and dominant matches found in previous rows, like in Algorithm 1. Analogously we can compute dominant matches (i, l), $i > l$, in column l with the aid of $CLOSEST\text{-}A$ and dominant matches found in previous columns. Information about dominant matches found in previous rows is recorded in a vector $row\text{-}Thresh$ containing for each class C_k the leftmost position of a dominant k-match found so far. Symmetrically $col\text{-}Thresh$ contains for each class the topmost position of a corresponding dominant match found in previous columns.

Having reached point (l, l) we can detect whether $l = l_{min}(k)$ for some class C_k, i.e. whether all dominant k-matches have been determined, by testing whether $l = row\text{-}Thresh[k]$ or $l = col\text{-}Thresh[k]$. Further notice that at each point (l, l) we have to perform this test only for one distinguished class C_{low} since no two classes may be completed at the same time; otherwise the contours of the classes would have to intersect or touch. In computing dominant matches in a certain row and column we therefore only have to look for dominant k-matches where $k \geq low$, thus saving checks of the values $row\text{-}Thresh[k']$ and $col\text{-}Thresh[k']$, $k' < low$, because all dominant k'-matches have been determined already.

Having in mind this discussion the following **Algorithm 2** [13] can easily be seen to implement exactly the ideas presented above. From this, the correctness follows immediately, too.

Algorithm 2

Input: Sequences $A = a_1 a_2 \ldots a_m$ and $B = b_1 b_2 \ldots b_n, m \leq n$.

Output: A LCS $S = s_1 s_2 \ldots s_p$.

Method:

(0) Preprocessing:
Compute $CLOSEST\text{-}A$ and $CLOSEST\text{-}B$
(1) Initialization:
$row\text{-}Thresh[0] := 0; col\text{-}Thresh[0] := 0;$
for $k := 1$ to m
do $col\text{-}Thresh[k] := m + 1; row\text{-}Thresh[k] := n + 1;$ **od**;
$low := 1;$
(2) **for** $l := 1$ to m **do** (* slide down diagonal *)
(3) **if** $col\text{-}Thresh[low] = l$ **then** (* all dom. matches found in C_{low} ? *)
$j := CLOSEST\text{-}B[a_l, row\text{-}Thresh[low]];$
$low := low + 1;$
else
$j := CLOSEST\text{-}B[a_l, l - 1];$
fi;
$k := low;$
(4) **while** $j \neq n + 1$ **do** (* find dom. matches in row l *)
if $j < row\text{-}Thresh[k]$ **then**
$temp := row\text{-}Thresh[k];$
$row\text{-}Thresh[k] := j;$
record new dominant match (l, j);
$j := CLOSEST\text{-}B[a_l, temp];$
fi;
if $j = row\text{-}Thresh[k]$ **then** $j := CLOSEST\text{-}B[a_l, j]$ **fi**;
$k := k + 1;$
od;
(5) **if** $row\text{-}Thresh[low] = l$ **then** (* all dom. matches found in C_{low} ? *)
$i := CLOSEST\text{-}A[b_l, col\text{-}Thresh[low]];$
$low := low + 1;$
else
$i := CLOSEST\text{-}A[b_l, l];$
fi;
$k := low;$
(6) **while** $i \neq m + 1$ **do** (* find dom. matches in column l *)
if $i < col\text{-}Thresh[k]$ **then**
$temp := col\text{-}Thresh[k];$
$col\text{-}Thresh[k] := i;$
record new dominant match (i, l);
$i := CLOSEST\text{-}A[b_l, temp];$
fi;
if $i = col\text{-}Thresh[k]$ **then** $i := CLOSEST\text{-}A[b_l, i]$ **fi**;
$k := k + 1;$
od;
od;
(7) recover LCS S

With respect to the time complexity, recall the discussion after the proof of the theorem which shows the upper bound of $O(p(n-p+1))$ for the computation of all dominant matches. A second upper bound is given by $O(pm)$ since at each of the m positions on the diagonal at most $O(p)$ *row-Thresh* and *col-Thresh* values will be checked. Taking into account the preprocessing we have an overall time complexity of $O(ns + \min\{pm, p(n-p)\})$. It is easy to verify the space complexity $O(ns + d)$, where d is the number of dominant matches.

4 Experimental Results

In order to validate the efficiency of the new algorithm, some experiments were performed that compared it to previous methods. As a measure of efficiency that is independent of the actual implementation, the number of checks of elements in vector(s) $Thresh$ was chosen (which is equivalent to the number of iterations of the main processing block in all algorithms using the paradigm of computing dominant matches). Omitting time $O(ns)$ for the preprocessing, the following six algorithms have been evaluated. First Algorithm 1 by Apostolico and Guerra [1] with running time $O(pm)$. Second the algorithm presented by Nakatsu et al. [11] which can be implemented in time $O(p(m-p))$ using the preprocessing introduced in Section 2. The original time bound was $O(n(m-p))$. Third the algorithm developed by Chin and Poon [2] with time complexity $O(\min\{pm, ds\})$. Fourth the new Algorithm 2 presented in this paper.

Also included in the comparison is another new algorithm which, in a certain sense, is equivalent to the Chin/Poon algorithm but uses a different order of computation and a different data structure. Again, the strategy is to avoid unnecessary checks of $Thresh$-values when searching dominant matches in a certain row. The main idea is as follows: from the calculation of dominant matches in row i_1 we can conclude that positions in B in intervals $[T_{i_1-1,k}+1, T_{i_1,k+1}-1]$ cannot match a_{i_1}. Now look at the smallest row $i_2 > i_1$ such that $a_{i_2} = a_{i_1}$. Then we can conclude from $T_{i_2-1,k} = T_{i_1-1,k}$ that $T_{i_2,k+1} = T_{i_2-1,k+1}$, i.e. we do not have to check $Thresh[k+1]$ in row i_2 because we know this value cannot change and so we can't find a new dominant $(k+1)$-match in row i_2. So $Thresh[k+1]$ may possibly change in row i_2 only if $Thresh[k]$ has been reduced in at least one of the rows $i_1, i_1+1, \ldots, i_2-1$. Thus, we have to remember in which row the individual $Thresh$-values changed for the last time. This is achieved by employing a doubly linked list. Each time an individual $Thresh$-value is reduced it is deleted at its old position and is inserted at the beginning of the list. Thus by scanning the list from the beginning we can always obtain those $Thresh$-values that changed most recently in preceding rows. This provides us exactly with the information needed to identify the $Thresh$-values that might change in the current row under consideration. For details on this algorithm see [13]. Since the underlying concepts of the two new algorithms are orthogonal to each other, they may also be used in combination, which was the last algorithm in the experiments.

The results are shown in Table 1 as the average of 100 pairs of input sequences. The new Algorithm 2 clearly beats the first two algorithms. Only the $O(ds)$ algorithms compare well for short longest common subsequences for quite a long time. The combined algorithm joins merits from both algorithms. Notice the very close approximation of dominant matches by the new Algorithm 2 for long longest common subsequences. This means that there are almost no superfluous checks of $Thresh$-values in this case.

Table 1. Frequency of checks of $Thresh$-values for six different algorithms. $|A| = 4000$, $|B| = 4000$, $|\Sigma| = 16$.

$\|LCS\|$	dominant matches	Apostolico/ Guerra $O(pm)$ **Algorithm 1**	Nakatsu et al. $O(p(m-p))$	Chin/Poon $O(ds)$	New $O(p(n-p))$ **Algorithm 2**	New $O(ds)$	Combined New $O(p(n-p))$
50	1,435	120,679	147,361	9,638	60,112	9,638	16,904
97	4,989	262,925	284,614	32,499	141,838	32,499	37,979
195	17,602	543,821	549,565	109,935	307,136	109,935	107,627
378	53,424	1,046,973	995,970	313,505	579,587	313,505	278,950
684	133,360	1,816,673	1,598,142	718,009	906,427	718,009	573,656
1,034	257,687	2,692,127	2,166,799	1,277,350	1,255,101	1,277,350	931,968
1,352	401,620	3,475,230	2,567,138	1,868,036	1,569,218	1,868,036	1,288,738
1,572	513,599	4,024,663	2,792,765	2,293,083	1,777,573	2,293,083	1,539,632
1,663	506,354	4,094,862	2,715,082	2,300,281	1,532,089	2,300,281	1,367,447
2,161	423,878	4,748,025	2,414,038	2,236,754	859,494	2,236,754	814,394
2,830	282,017	5,867,671	1,861,615	1,819,923	410,560	1,819,923	402,579
3,337	165,328	6,772,005	1,203,500	1,224,247	200,006	1,224,247	199,103
3,648	90,828	7,344,405	689,941	735,664	99,845	735,664	100,167
3,818	48,939	7,663,327	373,510	424,754	51,206	424,754	51,774
3,907	26,783	7,830,310	197,717	250,206	27,353	250,206	28,148

Comparing the actual running times of straightforward implementations of the algorithms resulted in Table 2. The algorithm by Wu et al. [17], using the paradigm of computing a shortest-path, seems to be the fastest for long longest common subsequences. So it was included in the comparison of running times, too. As can be seen, Algorithm 2 now even beats the Chin/Poon algorithm. The algorithm of Wu et al. proved to be very specialized. It is extremely fast for long longest common subsequences but its running time decreases dramatically for shorter ones. This is also due in part to a second specialization of this algorithm on situations when the input sequences differ in length considerably. However, in this case the sequences can't be similar, just because of their difference in length; remember that the application we have in mind was to find sequences in a database that are similar to a query.

Unfortunately, the combined algorithm is not as fast for longest common subsequences of intermediate length as it could be expected from Table 1. Here Algorithm 2 clearly performs best of all.

Table 2. Running times for seven algorithms in msec; $|A| = 4000$, $|B| = 4000$, $|\Sigma| = 16$.

$\|LCS\|$	Apostolico/ Guerra $O(pm)$ **Algorithm 1**	Nakatsu et al. $O(p(m-p))$	Chin/Poon $O(ds)$	New $O(p(n-p))$ **Algorithm 2**	New $O(ds)$	Combined New $O(p(n-p))$	Wu et al. $O(n(m-p))$
50	644	1,350	1,044	745	371	685	123,388
97	1,074	2,350	1,229	1,011	550	849	120,120
195	1,945	4,293	1,875	1,554	1,187	1,417	114,183
378	3,539	7,604	3,637	2,505	2,910	2,892	103,586
684	6,067	12,159	7,267	3,770	6,527	5,680	87,605
1,034	9,084	16,598	12,447	5,285	11,441	9,456	70,496
1,352	11,920	19,839	18,043	6,738	16,903	13,511	56,835
1,572	14,058	21,702	22,111	7,771	21,282	16,471	48,094
1,663	14,163	21,095	22,069	6,908	21,487	15,342	45,383
2,161	15,841	18,731	20,607	4,679	20,578	10,933	28,329
2,830	18,668	14,476	16,071	2,791	16,007	6,641	11,482
3,337	20,947	9,451	10,836	1,737	10,681	3,915	3,726
3,648	22,386	5,533	6,827	1,164	6,389	2,358	1,074
3,818	23,187	3,117	4,344	865	3,646	1,520	309
3,907	23,606	1,774	2,969	708	2,163	1,091	100

5 Conclusion

A new flexible algorithm for the LCS problem was presented which avoids a possibly difficult or even impossible a priori decision for some specialized algorithm. The new algorithm competes well with previous algorithms for both short and long longest common subsequences and can deal with longest common subsequences of intermediate length in a more efficient manner than any previous algorithm. The new algorithm is based on the well-known paradigm of computing dominant matches and was obtained through a kind of dualization. Its time and space complexity is $O(ns+\min\{pm, p(n-p)\})$ and $O(ns+d)$ respectively. It remains to give a linear space implementation of this algorithm and to generalize it to more than two sequences where the savings can be conjectured to be even larger.

Acknowledgment. I would like to thank Prof. Dr. N. Blum for drawing my attention to the LCS problem.

References

1. A. Apostolico, C. Guerra: *The Longest Common Subsequence Problem Revisited*, Algorithmica, Vol. **2**, 1987, 315–336.
2. F. Y. L. Chin, C. K. Poon: *A Fast Algorithm for Computing Longest Common Subsequences of Small Alphabet Size*, Journal of Information Processing, Vol. **13**, No. 4, 1990, 463–469.
3. F. Y. L. Chin, C. K. Poon: *Performance Analysis of Some Simple Heuristics for Computing Longest Common Subsequences*, Algorithmica, Vol. **12**, 1994, 293–311.

4. V. Dančík: *Expected Length of Longest Common Subsequences*, PhD thesis, University of Warwick, 1994.
5. D. Eppstein, Z. Galil, R. Giancarlo, G. F. Italiano: *Sparse Dynamic Programming I: Linear Cost Functions*, Journal of the ACM, Vol. **39**, No. 3, 1992, 519–545.
6. D. S. Hirschberg: *Algorithms for the Longest Common Subsequence Problem*, Journal ACM, Vol. **24**, Oct. 1977, 664–675.
7. W. J. Hsu, M. W. Du: *New Algorithms for the LCS Problem*, Journal of Computer and System Sciences **29**, 1984, 133–152.
8. J. W. Hunt, T. G. Szymanski: *A Fast Algorithm for Computing Longest Common Subsequences*, Comm. ACM, Vol. **20**, May 1977, 350–353.
9. W. J. Masek, M. S. Paterson: *A Faster Algorithm Computing String Edit Distances*, Journal of Computer and System Sciences **20**, 1980, 18–31.
10. E. W. Myers: *An $O(ND)$ Difference Algorithm and Its Variations*, Algorithmica, Vol. **1**, 1986, 251–266.
11. N. Nakatsu, Y. Kambayashi, S. Yajima: *A Longest Common Subsequence Algorithm Suitable for Similar Text Strings*, Acta Informatica **18**, 1982, 171-179.
12. M. Paterson, V. Dančík: *Longest Common Subsequences*, Proceedings of the 19th Intern. Symp. on Mathematical Foundations of Computer Science, Vol. **841** of LNCS, 1994, 127–142.
13. C. Rick: *New Algorithms for the Longest Common Subsequence Problem*, Research Report No. **85123-CS**, University of Bonn, 1994.
14. D. Sankoff, J. B. Kruskal (Ed.): *Time Warps, String Edits and Macromolecules: The Theory and Practice of Sequence Comparison*, Addison-Wesley: Reading, MA, 1983.
15. E. Ukkonen: *Algorithms for Approximate String Matching*, Information and Control **64**, 1985, 100–118.
16. R. A. Wagner, M. J. Fischer: *The String to String Correction Problem*, Journal ACM, Vol. **21**, 1974, 168–173.
17. S. Wu, U. Manber, G. Myers, W. Miller: *An $O(NP)$ Sequence Comparison Algorithm*, Information Processing Letters **35**, 1990, 317–323.

This article was processed using the LaTeX macro package with LLNCS style